

Planck Polarization Component Separation in STL

Alexandros Tsouros

Problem setup

We observe a Planck polarization patch:

$$d_Q, d_U \in \mathbb{R}^{H \times W}.$$

The goal is to recover two unknown signal maps

$$\tilde{s} = (\tilde{s}_Q, \tilde{s}_U),$$

using:

- the observed Q/U maps,
- paired nuisance samples $(n_Q^{(i)}, n_U^{(i)})$,
- a fixed auxiliary dust tracer d_I from l857.

What is optimized?

The optimization variable is the two-map tensor

$$u = \begin{bmatrix} u_Q \\ u_U \end{bmatrix} \in \mathbb{R}^{2 \times H \times W}.$$

Everything else is fixed during a minibatch step:

$$d_Q, d_U, d_I, \quad \{(n_Q^{(i)}, n_U^{(i)})\}_{i \in I}.$$

The code initializes u either from the data maps or from white noise with matching standard deviations. After optimization, the final recovered maps are denoted

$$\tilde{s}_Q, \tilde{s}_U.$$

Core idea: match scattering statistics

Define a feature map $\Phi(\cdot)$ from STL scattering statistics.

For a minibatch of nuisance indices I , compare:

$$y(I) = \Phi(X_{\text{target}}(I)), \quad \tilde{y}(u; I) = \Phi(X_{\text{run}}(u; I)).$$

The loss is simply

$$\mathcal{L}(u; I) = \|\tilde{y}(u; I) - y(I)\|_2^2.$$

Choosing noise realizations

The nuisance files are paired before optimization:

$(n_Q^{(j)}, n_U^{(j)})$ share the same noise/CMB-residual seed key.

At one optimization step, the script samples a set of indices

$$I = \{i_1, \dots, i_{N_b}\},$$

For those indices it loads

$$n_Q^I, n_U^I \in \mathbb{R}^{N_b \times H \times W}.$$

This leading dimension is the noise-realization dimension.

The batch dimension

Each selected realization is treated as one batch element. The maps that do not depend on the realization are broadcast along this dimension:

$$d_Q, d_U, d_I, u_Q, u_U \in \mathbb{R}^{H \times W} \longrightarrow \mathbb{R}^{N_b \times H \times W}.$$

After broadcasting fixed maps and inserting the sampled nuisance maps, the channels are stacked:

$$X_{\text{target}}(I), X_{\text{run}}(u; I) \in \mathbb{R}^{N_b \times C \times H \times W}.$$

Here $C = 5$ in the full phase and $C = 3$ in the optional warm-up phase.

Averaging over realizations

STL is applied to the whole batch tensor, not to one realization at a time:

$$X(I) \in \mathbb{R}^{N_b \times C \times H \times W} \longmapsto \Phi(X(I)).$$

The flattened statistics are then averaged along the batch dimension. So the loss compares the *mean* STL statistics over the selected realizations:

$$\bar{\Phi}_{\text{target}}(I) \quad \text{against} \quad \bar{\Phi}_{\text{run}}(u; I).$$

This means one optimizer step asks the same running maps u_Q, u_U to work across several nuisance realizations at once.

The target channels

For each selected nuisance pair $(n_Q^{(i)}, n_U^{(i)})$, the target batch element is

$$X_{\text{target}}^{(i)} = \begin{bmatrix} d_Q \\ n_Q^{(i)} \\ d_U \\ n_U^{(i)} \\ d_I \end{bmatrix} \in \mathbb{R}^{5 \times H \times W}.$$

Across all selected indices:

$$X_{\text{target}}(I) \in \mathbb{R}^{N_b \times 5 \times H \times W}.$$

The observed maps and auxiliary map are repeated across the batch; only the nuisance maps vary with i .

The running channels

For the current running maps $u = (u_Q, u_U)$, each running batch element is

$$\chi_{\text{run}}^{(i)}(u) = \begin{bmatrix} u_Q + n_Q^{(i)} \\ d_Q - u_Q \\ u_U + n_U^{(i)} \\ d_U - u_U \\ d_I \end{bmatrix}.$$

Across all selected noise realizations:

$$\chi_{\text{run}}(u; I) \in \mathbb{R}^{N_b \times 5 \times H \times W}.$$

Each polarization field is split into:

- a *data-like* channel: running map plus sampled nuisance,
- a *residual* channel: what remains in the observed map.

How fields are put into one channel

The first and third running channels are the key construction:

$$u_Q + n_Q^{(i)}, \quad u_U + n_U^{(i)}.$$

This deliberately makes one channel look like an observed polarization map:

structured signal field + one nuisance realization.

The optimizer changes the same u_Q, u_U for all batch elements. The only difference between batch elements is which nuisance realization is added.

The residual channels

The second and fourth channels are not sampled:

$$r_Q = d_Q - u_Q, \quad r_U = d_U - u_U.$$

They are repeated across the minibatch:

$$r_Q, r_U \in \mathbb{R}^{H \times W} \longrightarrow \mathbb{R}^{N_b \times H \times W}.$$

This lets STL see whether the part removed from the data behaves like the nuisance channels.

Auxiliary dust tracer

The I857 map is a fixed auxiliary channel:

$$d_I = I_{857} - \langle I_{857} \rangle.$$

It is included in both target and running tensors, unchanged:

$$X_{\text{target}} : d_I, \quad X_{\text{run}} : d_I.$$

Its role is to give the scattering statistics access to morphology correlated with polarized dust.

Which channel relations are measured?

The 5-channel cross matrix keeps only selected statistics:

$$\begin{pmatrix} 1 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{pmatrix}.$$

So STL measures:

- auto-statistics of all channels,
- Q–aux and U–aux relations,
- no explicit Q–U or signal–residual cross terms.

What ST_COMPUTE_PS adds

With `ST_COMPUTE_PS = True`, the feature map is enlarged:

$$\Phi(X) = [\text{mean}, \text{variance}, S_1, S_2, S_3, S_4, P].$$

The new object P is a radially binned cross-power spectrum of the same 5-channel tensor:

$$X^{(i)} = \begin{bmatrix} d_Q \\ n_Q^{(i)} \\ d_U \\ n_U^{(i)} \\ d_I \end{bmatrix} \quad \text{or} \quad X_{\text{run}}^{(i)}(u).$$

If `ST_COMPUTE_PS = False`, all scattering statistics are still computed, but this extra Fourier-domain power-spectrum block is omitted.

What power spectra are compared?

For a 5-channel minibatch,

$$P(X) \in \mathbb{C}^{N_b \times 5 \times 5 \times B},$$

where B is the number of radial Fourier bins.

The same cross matrix is reused, so only the selected channel pairs enter the loss:

$$(d_Q, d_Q), (d_Q, d_I), (n_Q, n_Q), (d_U, d_U), (d_U, d_I), (n_U, n_U), (d_I, d_I).$$

So the power-spectrum term asks the running 5-channel object to match the target 5-channel object also in radial Fourier power, not only in wavelet scattering structure.

How the code computes P

For each channel c , the FFT backend computes

$$\hat{X}_c(k).$$

For each radial bin b , it applies a Fourier mask $M_b(k)$ and forms

$$P_{c_1 c_2 b} = \left\langle M_b(k) \hat{X}_{c_1}(k) \overline{\hat{X}_{c_2}(k)} \right\rangle_k.$$

In the non-periodic case, the code transforms the binned maps back to real space, crops unreliable borders, and then averages the same channel-pair products.

Why this changes the loss

With `ST_COMPUTE_PS = True`, the loss contains one more constraint:

$$\mathcal{L}(u; l) = \|\bar{\Phi}_{\text{run}}(u; l) - \bar{\Phi}_{\text{target}}(l)\|_2^2.$$

The power-spectrum entries inside Φ constrain the two-point Fourier content of

$$[u_Q + n_Q, d_Q - u_Q, u_U + n_U, d_U - u_U, d_l].$$

The scattering entries still constrain the non-Gaussian, multi-scale, orientation-dependent morphology.

Memory estimate for backprop

Let the full 5-channel phase have

$$C = 5, \quad N_b = 15, \quad H = W = N, \quad B \simeq N/16.$$

For `float64`, complex tensors use 16 bytes per element. The first large object is the stack of binned maps:

$$X_{\text{bin}} \sim N_b \times C \times B \times N \times N.$$

In the non-periodic path, the code keeps both Fourier-binned maps and inverse-FFT binned maps. This gives the contribution

$$2 N_b C B N^2.$$

Where the C^2 term comes from

The expensive step is comparing channel pairs. From the same 5-channel object, the code forms products of channel c_1 against channel c_2 :

$$X_{\text{cross}} \sim N_b \times C \times C \times B \times N \times N.$$

The two C factors are the two channel indices in the cross spectrum:

$$P_{c_1 c_2 b}.$$

Again, in the non-periodic path there is a Fourier-side product and a real-space/cropped product, giving

$$2 N_b C^2 B N^2.$$

Combined memory estimate

Putting the binned-map and channel-pair terms together:

$$2 N_b C^2 B N^2 + 2 N_b C B N^2$$

complex numbers.

So the extra saved memory is roughly

$$16 (2 N_b C^2 B N^2 + 2 N_b C B N^2) = 900 N^3 \text{ bytes.}$$

The C^2 term is dominant because it stores all selected channel-pair products before the final average over pixels.

Numbers for the default setting

For the power-spectrum block alone:

N	$B \simeq N/16$	ST_COMPUTE_PS=False	ST_COMPUTE_PS=True
128	8	0	≈ 1.8 GiB
256	16	0	≈ 14.1 GiB
512	32	0	≈ 112.5 GiB
1024	64	0	≈ 900 GiB

The final coefficients are small:

$$N_b \times C \times C \times B.$$

The memory cost comes from the full-grid tensors needed to differentiate the power-spectrum computation.

Ways to reduce memory

The most direct reductions are to shrink the large power-spectrum tensors:

- set `ST_COMPUTE_PS=0` when the power-spectrum constraint is not essential,
- reduce `BATCH`, since memory scales linearly with N_b ,
- reduce `MAP_SIZE`, since the estimate scales roughly like N^3 ,
- use `DTYPE=float32`, which halves complex-tensor memory,
- reduce the L-BFGS `history_size`, which lowers the persistent optimizer state.

A code-level improvement would be to compute selected channel pairs or radial bins in smaller chunks, so the full

$$N_b \times C \times C \times B \times N \times N$$

object is never materialized at once.

Optional warm-up without explicit noise channels

The script can begin with a simpler 3-channel phase:

$$X_{\text{target}}^{(i)} = \begin{bmatrix} d_Q \\ d_U \\ d_I \end{bmatrix}, \quad X_{\text{run}}^{(i)}(u) = \begin{bmatrix} u_Q + n_Q^{(i)} \\ u_U + n_U^{(i)} \\ d_I \end{bmatrix}.$$

This phase matches data-like Q/U channels before introducing explicit residual channels.

Normalization and operators

Two STL operators are built:

Φ_3 for 3 channels, Φ_5 for 5 channels.

Each stores a fixed normalization reference once, then reuses it during optimization:

`store_ref` $\rightarrow$ `load_ref`.

Minibatch loop

At each optimization step:

- ① choose paired nuisance indices I ,
- ② build target statistics once,
- ③ build running statistics inside the closure,
- ④ take one L-BFGS optimizer step.

The target statistics are computed without gradients. The running statistics are computed inside the closure so gradients flow back to u_Q, u_U .

- The optimized running maps are u_Q, u_U ; the final maps are denoted $\tilde{s}_Q, \tilde{s}_U$.
- Target channels contain observed Q/U, sampled Q/U nuisance, and fixed l857.
- Running channels mix fields as $u_Q + n_Q$ and $u_U + n_U$.
- Residual channels $d_Q - u_Q$ and $d_U - u_U$ expose what the running maps leave behind.
- A minibatch is an extra leading dimension over selected nuisance realizations.
- The cross matrix decides which channel relations STL is allowed to match.